專題：混合常態參數估計與限制式條件的最大值問題

學號：411079060

姓名：石欽鈺

完成日期：2024/01/04

專題目標：

1. 理解混合常態模型： 從前只學習過一個常態，但市面上可能產出的數據結果會是一個以上的常態混合而成的，不只需要了解平均數、標準差、混合比例對函數分佈意義上的影響，更需知道他呈現的圖為何，藉此專題更深入理解，其基本概念、組成要素，以及如何用數學方法描述混合常態分佈。
2. 參數估計：學習如何使用統計方法（例如EM演算法）估計混合常態分佈的參數。這牽涉到適當的參數初始化、收斂條件的設定，以及如何處理數據，也針對樣本數大小來觀察估計的準確度，從實際程式操作中了解該演算法的內涵
3. 限制式條件的最大值：學習在有約束條件下優化目標函數，並了解怎麼樣的結果（例如：結果值最大）會是最優解，尋找優化技術並使用
4. 數據可視化：直觀地展示混合常態分佈的估計結果。包括直方圖、PDF曲線、表格（dataframe）、等高線圖、3D立體圖
5. 實際應用：將理論(數學函數概念)應用到實際數據集，並且嘗試推倒到程式碼能接受的地步，看到這些技能如何在真實世界的數據分析中產生影響。 </font>

題目一：混合常態參數估計 Normal Mixture 作品目標：

- (1) 觀察不同混合比例、平均值、標準差設定之下，混合分佈會呈現何形狀
- (2) 學習如何定義混合常態的機率密度函數
- (3) 善用直方圖來表示他們混合的狀況
- (4) 使用最大概似估計法（最小化負的對數聯合概似函數）估計混合常態分佈的參數
- (5) 定義對數聯合概似函數，並自行選擇適合的參數（對參數的探討）來生成樣本
- (6) 繪製真實、估計、演算法（EM）推導的混合常態 PDF
- (7) 探討樣本數對估計值的影響
- (8) 學習 sklearn.mixture.GaussianMixture 的使用及初始值猜測

1. **設定混合分佈的參數：**
   - π_1: 混合分佈中第一個成分的比例
   - α_1, β_1: 混合分佈中第一個成分的 Beta 分佈的形狀參數
   - α_2, β_2: 混合分佈中第二個成分的 Beta 分佈的形狀參數
2. **畫出混合分佈的概率密度函數（PDF）：**
   - 使用混合分佈的參數（上述）計算並畫出混合分佈的 PDF
3. **生成模擬樣本：**
   - 從混合分佈中生成 n 個樣本
4. **進行最大概似估計（MLE）：**

- 定義對數聯合概似函數 L，搭配參數
- 使用 `opt.minimize` 函數，最小化對數聯合概似函數
- 使用的參數範圍、初始猜測值和優化選項都被設定好

5. **顯示估計結果：**

- 顯示最大概似估計的結果，包括 估計的參數值 和 對數聯合概似函數的極小值
- 印出最大概似估計的 結果與真實參數 之比較

函數使用

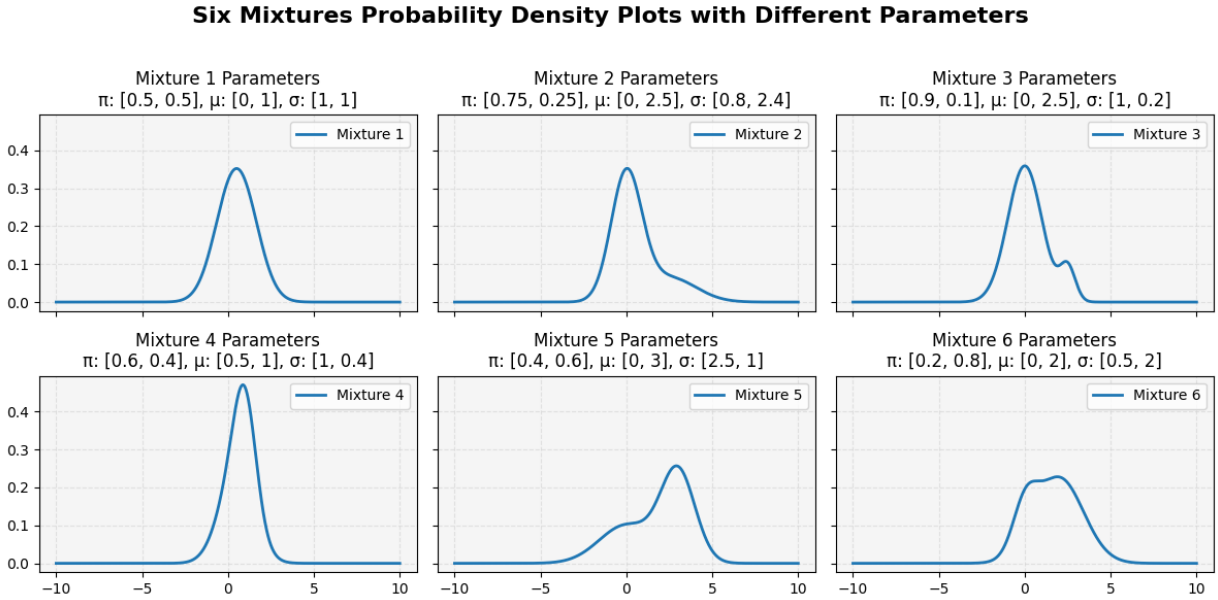
- 共享座標軸：
  - 目的：使用相同的座標軸刻度和範圍，以確保它們在軸上對齊
  - sharex=True / sharey=True
- 圖片背景：
  - set_facecolor('whitesmoke')
- 調整子圖比例：
  - tight_layout(rect=[0, 0, 1, 0.96])

- 圖 1-1：自行調整參數，來設定不同的混合樣本，觀察調整每個參數會影響的圖形層面，以及混合後的長相

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義六種混合常態的參數
params_list = [
    {'pi': [0.5, 0.5], 'mu': [0, 1], 'sigma': [1, 1]},
    {'pi': [0.75, 0.25], 'mu': [0, 2.5], 'sigma': [0.8, 2.4]},
    {'pi': [0.9, 0.1], 'mu': [0,2.5], 'sigma': [1, 0.2]},
    {'pi': [0.6, 0.4], 'mu': [0.5, 1], 'sigma': [1, 0.4]},
    {'pi': [0.4, 0.6], 'mu': [0, 3], 'sigma': [2.5, 1]},
    {'pi': [0.2, 0.8], 'mu': [0, 2], 'sigma': [0.5, 2]},
]

fig, axes = plt.subplots(2, 3, figsize=(12, 6), sharex=True, sharey=True)

for i, params in enumerate(params_list):
    ax = axes[i // 3, i % 3] # 計算子圖
    x_range = np.linspace(-10, 10, 1000)
    ax.plot(x_range, mix_pdf(x_range, params), label=f'Mixture {i+1}', linewi
    ax.set_title(f'Mixture {i+1} Parameters\nπ: {params["pi"]}, μ: {params["m
    ax.legend()

plt.suptitle('Six Mixtures Probability Density Plots with Different Parameter

# 背景顏色和隔線設定
for ax in axes.flat:
    ax.grid(True, linestyle='--', alpha=0.6, color='lightgrey')
    ax.set_facecolor('whitesmoke')
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96])  # 調整布局以避免擋住標題
plt.show()
```

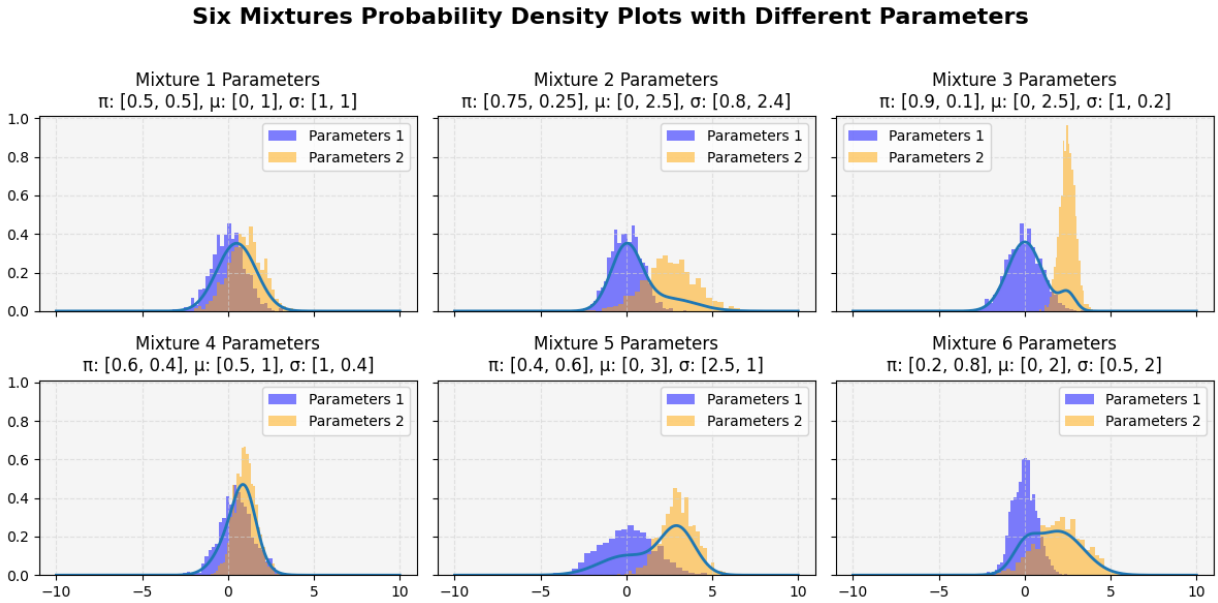


- 圖 1-2：用不同顏色畫兩種參數設定的直方圖

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義六種混合常態的參數
params_list = [
    {'pi': [0.5, 0.5], 'mu': [0, 1], 'sigma': [1, 1]},
    {'pi': [0.75, 0.25], 'mu': [0, 2.5], 'sigma': [0.8, 2.4]},
    {'pi': [0.9, 0.1], 'mu': [0,2.5], 'sigma': [1, 0.2]},
    {'pi': [0.6, 0.4], 'mu': [0.5, 1], 'sigma': [1, 0.4]},
    {'pi': [0.4, 0.6], 'mu': [0, 3], 'sigma': [2.5, 1]},
    {'pi': [0.2, 0.8], 'mu': [0, 2], 'sigma': [0.5, 2]},
]

# 繪製六張子圖
fig, axes = plt.subplots(2, 3, figsize=(12, 6), sharex=True, sharey=True)

for i, params in enumerate(params_list):
    ax = axes[i // 3, i % 3]
    x_range = np.linspace(-10, 10, 1000)
    ax.plot(x_range, mix_pdf(x_range, params), linewidth=2)

    # 添加兩個常態分佈的直方圖
    ## 將隨機樣本分成30個區間（bins），並計算每個區間中的樣本數
    ## bin_edges 區間邊界（起始點＆結束點）--> 畫圖會用到先保留起來
    hist1, bin_edges1 = np.histogram(np.random.normal(params['mu'][0], np.sqr
    hist2, bin_edges2 = np.histogram(np.random.normal(params['mu'][1], np.sqr

    ## bin_centers 區間中心：起始點和結束點相加，然後除以2
    ## [:-1] 選取除了最後一個元素之外的所有元素，這是為了避免超出邊界
    ## [1:] 選取除了第一個元素之外的所有元素
    bin_centers1 = (bin_edges1[:-1] + bin_edges1[1:]) / 2
```

```
    bin_centers2 = (bin_edges2[:-1] + bin_edges2[1:]) / 2

    ax.bar(bin_centers1, hist1, width=bin_edges1[1] - bin_edges1[0], alpha=0.
    ax.bar(bin_centers2, hist2, width=bin_edges2[1] - bin_edges2[0], alpha=0.

    ax.set_title(f'Mixture {i+1} Parameters\nπ: {params["pi"]}, μ: {params["m
    ax.legend()

plt.suptitle('Six Mixtures Probability Density Plots with Different Parameter

for ax in axes.flat:
    ax.grid(True, linestyle='--', alpha=0.6, color='lightgrey')
    ax.set_facecolor('whitesmoke')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



從以上兩圖可知，可能乍看之下並非常態的圖，卻可以用兩個以上的常態樣本來解釋出，有些高度重疊，有些只有擦邊，sigma會影響常態高聳或平坦，而平均數則影響常態的位置。

圖 2-1: 混合狀況一（較分散）的機率密度圖，函真實的與估計的pdf

- 注意事項：觀察線估計線是否越來越穩合真實

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import minimize
from sklearn.mixture import GaussianMixture

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義對數聯合概似函數
def log_likelihood(params, data):
    pi1, mu1, sigma1, mu2, sigma2 = params
    params_dict = {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2], 'sigma': [sigma1**
    pdf_values = mix_pdf(data, params_dict)
    log_likelihood = np.sum(np.log(pdf_values))
    return -log_likelihood
```

```
# 產生資料
np.random.seed(42)
mu1, mu2 = 0, 3
sigma1, sigma2 = 2.5, 1
pi1 = 0.6
sample_sizes = [50, 100, 300, 500, 1000, 10000]

fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for i, n in enumerate(sample_sizes):
    data = np.concatenate([np.random.normal(mu1, sigma1, int(n * pi1)),
                           np.random.normal(mu2, sigma2, int(n * (1 - pi1)))]

    # 使用 minimize 估計參數
    ## 放入 initial_params 初始猜測值（包含五個參數）
    ## args=(data,)：將數據 data 傳遞給 log_likelihood 函數
    ## Nelder-Mead 優化法：直接搜索，不需要梯度信息，通常用於少數變數的非線性最小化問題
    ### 梯度：多變數函數對每個變數的偏導數構成; 方向：函數在該點上增長最快的方向; 大小：增
    initial_params = [0.5, 2, 1, 5, 1]
    result = minimize(log_likelihood, initial_params, args=(data,), method='N
    estimated_params = result.x
    params_dict = {'pi': [estimated_params[0], 1 - estimated_params[0]],
                   'mu': [estimated_params[1], estimated_params[3]],
                   'sigma': [estimated_params[2]**2, estimated_params[4]**2]}

    # 設定直方圖漸層顏色
    color = plt.cm.viridis(i / len(sample_sizes))

    # 繪製直方圖
    ax = axes[i // 3, i % 3]
    ax.hist(data, bins=30, density=True, alpha=0.4, label='Data Histogram', c

    # 繪製真實的混合常態 PDF
    x_range = np.linspace(np.min(data), np.max(data), 1000)
    ax.plot(x_range, mix_pdf(x_range, {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2]
            label='True Mixture', linewidth=3, color='blue')

    # 繪製估計的混合常態 PDF
    ax.plot(x_range, mix_pdf(x_range, params_dict), label='Estimated Mixture'

    ax.set_title(f'Sample Size: {n}')
    ax.legend()

plt.suptitle('Mixture Estimation with Different Sample Sizes', fontsize=20, f
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

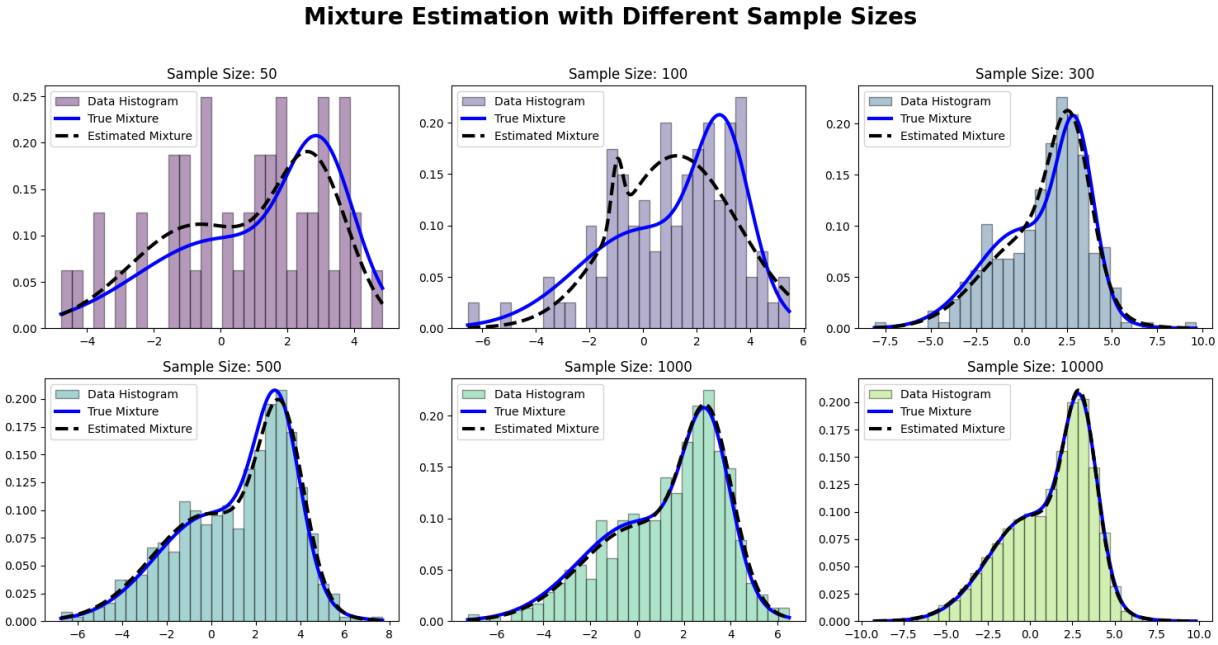


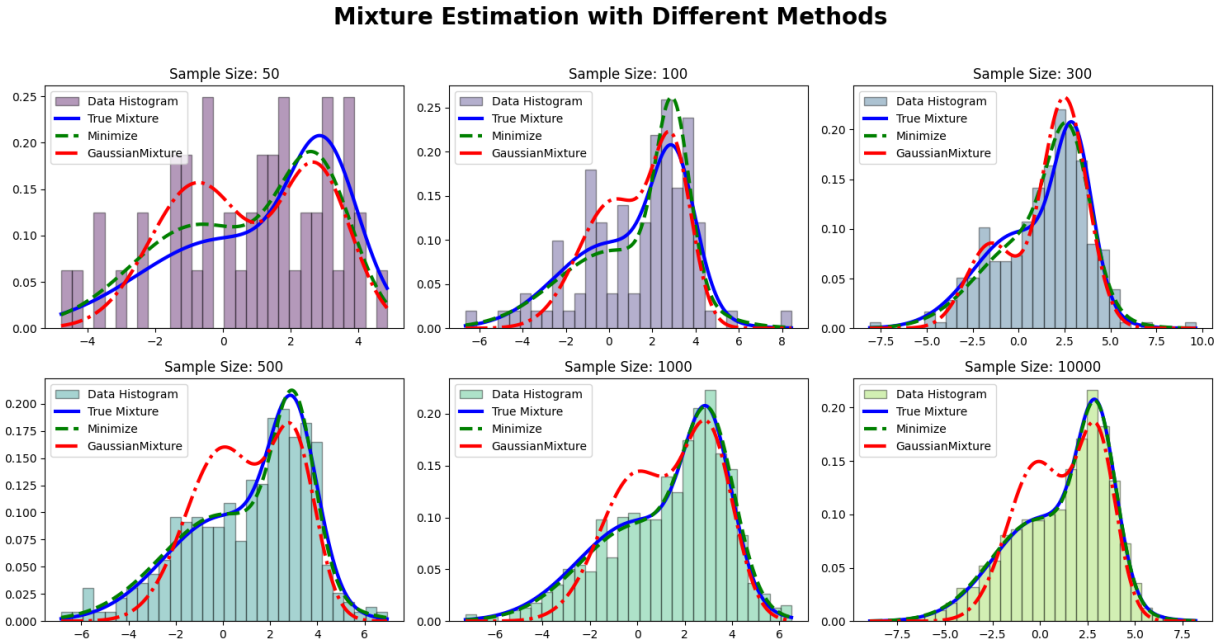
圖 3-1: 新增使用 EM 演算法估計

- 步驟
  - 生成合成數據：
    - 用 np.random.normal 生成具有兩個正態分佈混合的合成數據
    - 用 np.concatenate 將它們串聯
  - 擬合 GMM：
    - 用 scikit-learn 中的 GaussianMixture 類來擬合數據
    - fit 方法應用於整形後的數據（data.reshape(-1, 1)）
    - 指定參數: n_components、covariance_type、max_iter 和 tol
  - 提取參數：擬合 GMM 後，從模型中提取參數（weights、*means*、covariances_）
  - 結果呈現：直方圖、真實混合分佈、估計混合分佈以及使用 EM 演算法估計的混合分佈
- 演算法：EM 演算法對於估計潛在分佈的性能取決於多種因素
  - 數據量
  - 初始參數的選擇 init_params

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import minimize
from sklearn.mixture import GaussianMixture

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義對數聯合概似函數
def log_likelihood(params, data):
    pi1, mu1, sigma1, mu2, sigma2 = params
    params_dict = {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2], 'sigma': [sigma1**
    pdf_values = mix_pdf(data, params_dict)
    log_likelihood = np.sum(np.log(pdf_values))
    return -log_likelihood

# 產生資料
np.random.seed(42)
```

```
mu1, mu2 = 0, 3
sigma1, sigma2 = 2.5, 1
pi1 = 0.6
sample_sizes = [50, 100, 300, 500, 1000, 10000]

fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for i, n in enumerate(sample_sizes):
    data = np.concatenate([np.random.normal(mu1, sigma1, int(n * pi1)),
                           np.random.normal(mu2, sigma2, int(n * (1 - pi1)))]

    # 使用 optimize.minimize 估計參數
    initial_guess = [0.5, 0, 1, 3, 1]  # 隨意的初始猜測值
    result = minimize(log_likelihood, initial_guess, args=(data,), bounds=[(0
    params_minimize = result.x

    # 使用 GaussianMixture 估計參數
    gmm = GaussianMixture(n_components=2, covariance_type='full', verbose=0,
    gmm.fit(data.reshape(-1, 1))
    params_gmm = {'pi': gmm.weights_, 'mu': gmm.means_.flatten(), 'sigma': np

    # 設定直方圖漸層顏色
    color = plt.cm.viridis(i / len(sample_sizes))

    # 繪製直方圖
    ax = axes[i // 3, i % 3]
    ax.hist(data, bins=30, density=True, alpha=0.4, label='Data Histogram', c

    # 繪製真實的混合常態 PDF
    x_range = np.linspace(np.min(data), np.max(data), 1000)
    true_params = {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2], 'sigma': [sigma1**
    ax.plot(x_range, mix_pdf(x_range, true_params),
            label='True Mixture', linewidth=3, color='blue')

    # 繪製 optimize.minimize 估計的混合常態 PDF
    ax.plot(x_range, mix_pdf(x_range, {'pi': [params_minimize[0], 1 - params_
                                       'mu': [params_minimize[1], params_min
                                       'sigma': [params_minimize[2]**2, para
            label='Minimize', linestyle='--', linewidth=3, color='green')

    # 繪製 GaussianMixture 估計的混合常態 PDF
    ax.plot(x_range, mix_pdf(x_range, params_gmm), label='GaussianMixture', l

    ax.set_title(f'Sample Size: {n}')
    ax.legend()

plt.suptitle('Mixture Estimation with Different Methods', fontsize=20, fontwe
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



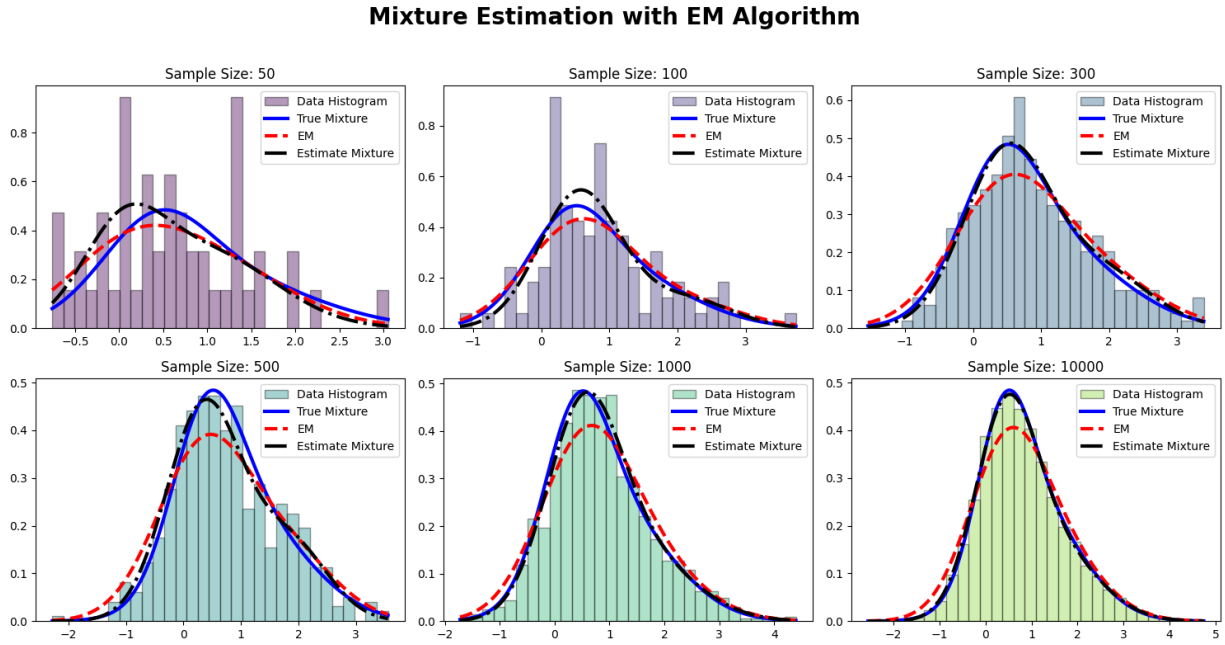
- 從圖中可以觀察出以下幾點：
  - 隨著樣本量的增加，兩種方法的結果都更加逼近真實混合分佈。
  - 樣本量較小（50、100）的情況下，存在較大的偏差; 樣本量較大（500、1000、10000）的情況下，較接近真實混合分佈。
- 注意事項：
  - Q: 為什麼我的程式碼看起來無誤，但EM的線卻沒有很準確
  - A:
    - 隨機初始化：EM 演算法對於初始值非常敏感。由於 GMM 通常使用隨機初始化的策略，可能導致不同的初始值，進而導致 EM 估計的混合模型分布有所不同。
      - 嘗試使用不同的隨機種子或改變初始值
    - 最大迭代次數：可能需要更多的迭代才能收斂
    - 數據分佈：如果實際數據分佈具有一些特殊形狀，或非常離散，EM 估計的 GMM 可能不夠準確。
      - 確保數據符合混合常態分佈的假設
    - 模型設置：GMM 模型設置為兩個組件 (n_components=2)，這可能不夠多，特別是當數據包含更多混合分佈時
      - 嘗試增加組件數量，查看結果是否改善

圖 2-2: 混合狀況二（較密集）的機率密度圖，函真實的與估計的 pdf & 使用 EM 演算法估計的線

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import minimize
from sklearn.mixture import GaussianMixture

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義對數聯合概似函數
```

```
def log_likelihood(params, data):
    pi1, mu1, sigma1, mu2, sigma2 = params
    params_dict = {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2], 'sigma': [sigma1**
    pdf_values = mix_pdf(data, params_dict)
    log_likelihood = np.sum(np.log(pdf_values))
    return -log_likelihood

# 產生資料
np.random.seed(42)
mu1, mu2 = 0.4, 1.2
sigma1, sigma2 = 0.6, 1
pi1 = 0.5
sample_sizes = [50, 100, 300, 500, 1000, 10000]

fig, axes = plt.subplots(2, 3, figsize=(15, 8))


# np.concatenate: 兩組數據串聯
# 生成樣本數為 int(n * pi1)
for i, n in enumerate(sample_sizes):
    data = np.concatenate([np.random.normal(mu1, sigma1, int(n * pi1)),
                           np.random.normal(mu2, sigma2, int(n * (1 - pi1)))]

    # 使用 EM 演算法估計參數
    gmm = GaussianMixture(n_components=2, covariance_type='full', verbose=0,
    gmm.fit(data.reshape(-1, 1))
    params_gmm = {'pi': gmm.weights_, 'mu': gmm.means_.flatten(), 'sigma': np

    # 設定直方圖漸層顏色
    color = plt.cm.viridis(i / len(sample_sizes))

    # 繪製直方圖
    ax = axes[i // 3, i % 3]
    ax.hist(data, bins=30, density=True, alpha=0.4, label='Data Histogram', c

    # 繪製真實的混合常態 PDF
    x_range = np.linspace(np.min(data), np.max(data), 1000)
    true_params = {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2], 'sigma': [sigma1**
    ax.plot(x_range, mix_pdf(x_range, true_params),
            label='True Mixture', linewidth=3, color='blue')

    # 繪製 EM 演算法估計的混合常態 PDF
    ax.plot(x_range, mix_pdf(x_range, params_gmm), label='EM', linestyle='--'


    # 額外加入 estimate mixture 圖二有的線條
    params_estimate = [params_gmm['pi'][0], params_gmm['mu'][0], params_gmm['
                       params_gmm['mu'][1], params_gmm['sigma'][1]**2]
    estimate_pdf_values = mix_pdf(x_range, {'pi': [params_estimate[0], 1 - pa
                                            'mu': [params_estimate[1], param
                                            'sigma': [params_estimate[2], pa
    ax.plot(x_range, estimate_pdf_values, label='Estimate Mixture', linestyle

    ax.set_title(f'Sample Size: {n}')
    ax.legend()


plt.suptitle('Mixture Estimation with EM Algorithm', fontsize=20, fontweight=
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



圖四：用表格呈現兩種不同方法估計雙變量函數參數的結果

1. 表格表達：使用兩種不同方法估計雙變量函數參數的結果
   - 方法 1：optimize.minimize
   - 方法 2：GaussianMixture
2. 表格設定：
   - n 列表示樣本數
   - pi1 和 pi2 列表示兩個混合分布的概率
   - mu1 和 mu2 列表示兩個混合分布的均值
   - s1 和 s2 列表示兩個混合分布的標準差
3. 結果顯示
   - optimize.minimize() 隨樣本增加，估計的參數逐漸收斂到真實值
   - GaussianMixture 與 optimize.minimize() 函數的結果非常相似
4. 結論：兩種方法都可以有效地估計雙變量函數參數，且隨著樣本增加，兩種方法的估計都變得越來越準確

函數介紹：

- n_components=2: 擬合的混合模型中有兩個成分（即兩個高斯分布）
- covariance_type='full': 每個成分都有其自己的協方差矩陣，即完整的協方差矩陣
- verbose=0: 不輸出詳細運行信息
- `max_iter=1000`：EM（期望最大化）算法的最大迭代次數
  - 為什麼要設置最大迭代次數？ 確保算法不會無窮迭代下去（所謂收斂）--> 控制計算效率、避免無窮迴圈
- `tol=1e-6`：EM 算法的 收斂容忍 度，當兩次迭代之間的對數似然差異小於此值時，算法將停止

- gmm.fit(data.reshape(-1, 1)):

1. 將資料轉換為一列，因為 GMM 預期每一列是一個樣本，每一列的數據是一個維度
2. 使用 EM 算法擬合 GMM 模型到資料

- flatten(): 將多維數組轉換成一維形式

- pd.concat 連接(concatenate)兩個 DataFrame
  - 將一行新的結果 row_minimize 與 columns_minimize 合併到 results_df_minimize 中，並使用 ignore_index=True 重新索引結果 DataFrame
  - 每次迭代結果都添加到 DataFrame

- Pandas 資料結構

1. Series： 一維，可以包含任意類型的數據
   - 類似 NumPy 一維陣列，但帶有標籤索引
2. DataFrame： 二維，表格型，由行列組成，每列可是不同的數據類型
   - 可看作是一個電子試算表或 SQL 表格

- Pandas 的功能

1. 數據清理：處理缺失數據、重複數據和異常值
2. 數據讀寫：多種格式的數據讀寫，如 CSV、Excel、SQL
3. 數據統計和計算：內建函數，如平均值、標準差、總和
4. 時間序列分析：支援日期和時間數據的處理
5. 數據合併和連接：合併不同數據集，類似 SQL 中的 JOIN 操作
6. 繪圖和可視化：整合 Matplotlib

In [ ]:

```
import numpy as np
import pandas as pd
from scipy.stats import norm
from scipy.optimize import minimize
from sklearn.mixture import GaussianMixture

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義對數聯合概似函數
def log_likelihood(params, data):
    pi1, mu1, sigma1, mu2, sigma2 = params
    params_dict = {'pi': [pi1, 1 - pi1], 'mu': [mu1, mu2], 'sigma': [sigma1**
    pdf_values = mix_pdf(data, params_dict)

    # 檢查是否有無效值
    if np.any(np.isnan(pdf_values)) or np.any(pdf_values <= 0):
        return np.inf

    log_likelihood = np.sum(np.log(pdf_values))
    return -log_likelihood

# 設定參數
np.random.seed(100)
mu1, mu2 = 0, 3
sigma1, sigma2 = 2.5, 1
pi1 = 0.6
sample_sizes = [50, 100, 300, 500, 1000, 10000]

# 初始化表格
columns_minimize = ['n', 'pi1(0.6)', 'pi2(0.4)', 'mu1(0)', 's1(2.5)', 'mu2(3)
                    's2(1)']
```

```
columns_gmm = ['n', 'pi1(0.6)', 'pi2(0.4)', 'mu1(0)', 's1(2.5)', 'mu2(3)',
                    's2(1)']

# 創建空的 DataFrame 用於存放 optimize.minimize 的結果
results_df_minimize = pd.DataFrame(columns=columns_minimize)

# 創建空的 DataFrame 用於存放 GaussianMixture 的結果
results_df_gmm = pd.DataFrame(columns=columns_gmm)

# 開始迭代不同的樣本數
for n in sample_sizes:
    data = np.concatenate([np.random.normal(mu1, sigma1, int(n * pi1)),
                           np.random.normal(mu2, sigma2, int(n * (1 - pi1)))]

    # 使用 optimize.minimize 估計參數
    initial_guess = [0.55, 0, 2.4, 3, 1]  # 隨意的初始猜測值
    result_minimize = minimize(log_likelihood, initial_guess, args=(data,), b
    params_minimize = result_minimize.x

    # 使用 GaussianMixture 估計參數
    gmm = GaussianMixture(n_components=2, covariance_type='full', verbose=0,
    gmm.fit(data.reshape(-1, 1))
    params_gmm = {'n': n, 'pi1': gmm.weights_[0], 'pi2': gmm.weights_[1],
                  'mu1': gmm.means_[0][0], 'mu2': gmm.means_[1][0],
                  's1': np.sqrt(gmm.covariances_[0][0]), 's2': np.sqrt(gmm.c

    # 將 optimize.minimize 的結果加入表格
    row_minimize = [n, params_minimize[0], 1 - params_minimize[0], params_min
    results_df_minimize = pd.concat([results_df_minimize, pd.DataFrame([row_m

    # 將 GaussianMixture 的結果加入表格
    row_gmm = [params_gmm['n'], params_gmm['pi1'], params_gmm['pi2'], params_
    results_df_gmm = pd.concat([results_df_gmm, pd.DataFrame([row_gmm], colum

# 顯示結果
print("Results from optimize.minimize:")
print(results_df_minimize)

print("\nResults from GaussianMixture:")
print(results_df_gmm)
```

```
Results from optimize.minimize:
       n  pi1(0.6)  pi2(0.4)    mu1(0)   s1(2.5)    mu2(3)     s2(1)
0     50  0.406429  0.593571 -0.481212  1.734826  2.531912  1.105719
1    100  0.455534  0.544466 -1.117361  2.656870  2.843944  1.225913
2    300  0.708780  0.291220  0.712562  2.680041  3.084029  0.740704
3    500  0.640183  0.359817  0.322724  2.659425  2.893728  0.878272
4   1000  0.645440  0.354560  0.274360  2.478223  3.052643  0.920439
5  10000  0.589893  0.410107 -0.057276  2.472696  3.009352  1.025941

Results from GaussianMixture:
       n  pi1(0.6)  pi2(0.4)    mu1(0)                s1(2.5)    mu2(3)  \
0     50  0.396506  0.603494 -0.536459  [1.7105345993857985]  2.518660
1    100  0.547530  0.452470  2.839276  [1.2296468737170378] -1.138540
2    300  0.719501  0.280499  2.569690  [1.6731522132135879] -1.588916
3    500  0.362602  0.637398  2.892071  [0.8841212498542161]  0.312435
4   1000  0.641631  0.358369  0.259880  [2.4760045653125515]  3.049044
5  10000  0.585319  0.414681 -0.078549  [2.4668572928262407]  3.005590

                   s2(1)
0  [1.1114720122168102]
1  [2.6501888198199044]
2   [1.798169658791782]
3  [2.6589476550454383]
4  [0.9278004928101111]
5  [1.0332560130660735]
```

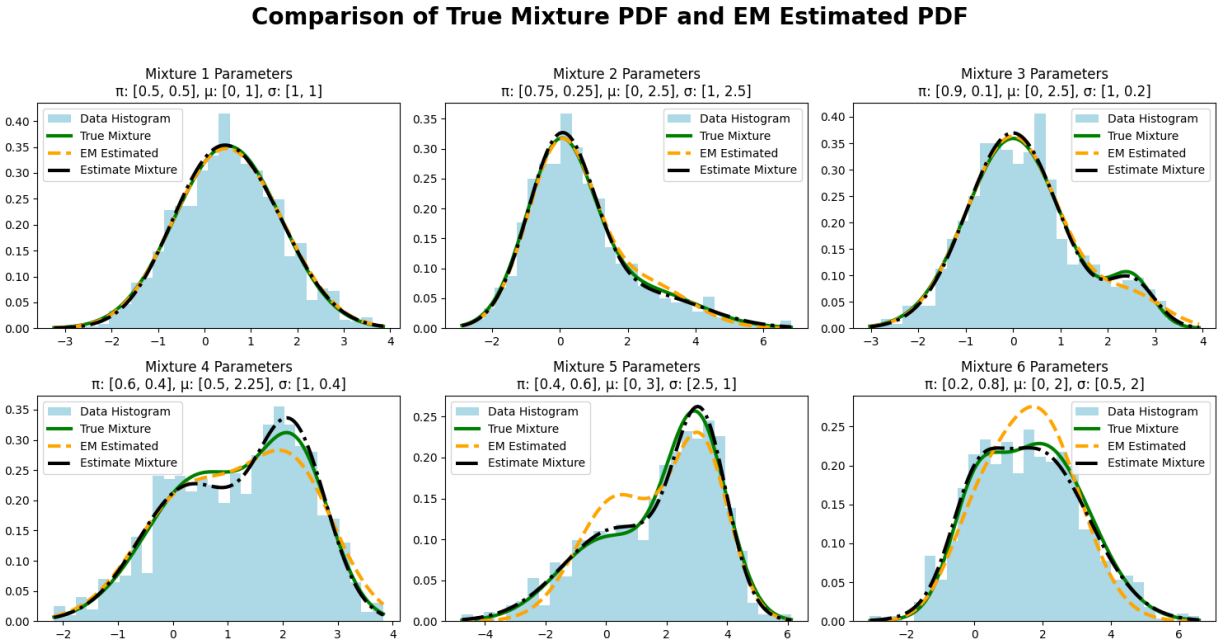
- 從圖中可以觀察出以下幾點：
  - 隨著樣本量的增加，兩種方法的參數估計都越來越趨近真實的設定，只會有小數點後幾位的些微差異
  - 整體來看，optimize.minimize 在樣本數較小時的估計準確度較 GaussianMixture 還要好，當然樣本數越大，估計的結果就越穩定也越準確
- 注意事項
  - 檢查是否有無效值 --> 用的話用對數轉換
    - if np.any(np.isnan(pdf_values)) or np.any(pdf_values <= 0): ;return np.inf
    - log_likelihood = np.sum(np.log(pdf_values))

圖五（補充）：在樣本數為 1000 的情況下，不同混合條件的在不同估計方法下與真實樣貌的差距

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from sklearn.mixture import GaussianMixture

# 定義混合常態的機率密度函數
def mix_pdf(x, params):
    pdf1 = norm.pdf(x, loc=params['mu'][0], scale=np.sqrt(params['sigma'][0])
    pdf2 = norm.pdf(x, loc=params['mu'][1], scale=np.sqrt(params['sigma'][1])
    return params['pi'][0] * pdf1 + params['pi'][1] * pdf2

# 定義 EM 演算法估計函數
def fit_gmm(data):
    gmm = GaussianMixture(n_components=2, covariance_type='full', verbose=0,
    gmm.fit(data.reshape(-1, 1))
    params_gmm = {'pi': gmm.weights_, 'mu': gmm.means_.flatten(), 'sigma': np
    return params_gmm

# 圖一給的參數
params_list = [
   {'pi': [0.5, 0.5], 'mu': [0, 1], 'sigma': [1, 1]},
    {'pi': [0.75, 0.25], 'mu': [0, 2.5], 'sigma': [1, 2.5]},
    {'pi': [0.9, 0.1], 'mu': [0,2.5], 'sigma': [1, 0.2]},
    {'pi': [0.6, 0.4], 'mu': [0.5, 2.25], 'sigma': [1, 0.4]},
    {'pi': [0.4, 0.6], 'mu': [0, 3], 'sigma': [2.5, 1]},
    {'pi': [0.2, 0.8], 'mu': [0, 2], 'sigma': [0.5, 2]},
]

# 產生資料並進行 EM 演算法估計
np.random.seed(42)
sample_size = 1000

fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for i, params in enumerate(params_list):
    data = np.concatenate([np.random.normal(params['mu'][0], np.sqrt(params['
                           np.random.normal(params['mu'][1], np.sqrt(params['

    params_gmm = fit_gmm(data)
    ax = axes[i // 3, i % 3]
    ax.hist(data, bins=30, density=True, alpha=1, label='Data Histogram', col

    # 繪製真實的混合常態 PDF
    x_range = np.linspace(np.min(data), np.max(data), 1000)
```

```
    ax.plot(x_range, mix_pdf(x_range, params), label='True Mixture', linewidt

    # 繪製 EM 演算法估計的混合常態 PDF
    ax.plot(x_range, mix_pdf(x_range, params_gmm), label='EM Estimated', line

    # 添加 EM 估計的混合 PDF
    params_estimate = [params_gmm['pi'][0], params_gmm['mu'][0], params_gmm['
                       params_gmm['mu'][1], params_gmm['sigma'][1]**2]
    estimate_pdf_values = mix_pdf(x_range, {'pi': [params_estimate[0], 1 - pa
                                            'mu': [params_estimate[1], param
                                            'sigma': [params_estimate[2], pa
    ax.plot(x_range, estimate_pdf_values, label='Estimate Mixture', linestyle

    ax.set_title(f'Mixture {i+1} Parameters\nπ: {params["pi"]}, μ: {params["m
    ax.legend()

plt.suptitle('Comparison of True Mixture PDF and EM Estimated PDF', fontsize=
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



- 注意事項
  - 可以發現演算法在某些圖中是很準確的，然而在某些圖形中卻有較大的誤差，相對 optimize.minimize 的估計準確度還要不穩定

題目：限制式條件的最大值問題 Constraint optimization

- 作品目標：進行最大概似估計（Maximum Likelihood Estimation, MLE）的優化過程，針對給定的資料擬合出最合適的參數值，也就是選擇 α, β 使得模型與實際資料的概似性最大（估計的模型最符合實際觀測到的數據分佈），兩個參數都需大於 0。採用了一個由兩個參數（alpha 和 beta）組成的模型，該模型基於一個給定的機率分佈函數。而找出最大值的方式為，善用最小值與最大值之間的關係，求出最小值來獲得最大值，此時 Negative Log-Likelihood 被用作目標函數，並由 log_likelihood 函數計算。常用時機：醫學統計上的存活問題

- 步驟：

1. 讀取 'UV.txt' 檔案，內含兩行資料 u 和 v，ui.vi是兩個樣本，樣本數=n，有兩個函數
2. 推導數學式到可程式化的地步 $\ln L(\alpha, \beta)$，其中 $\ln L$ 表示對數概似函數

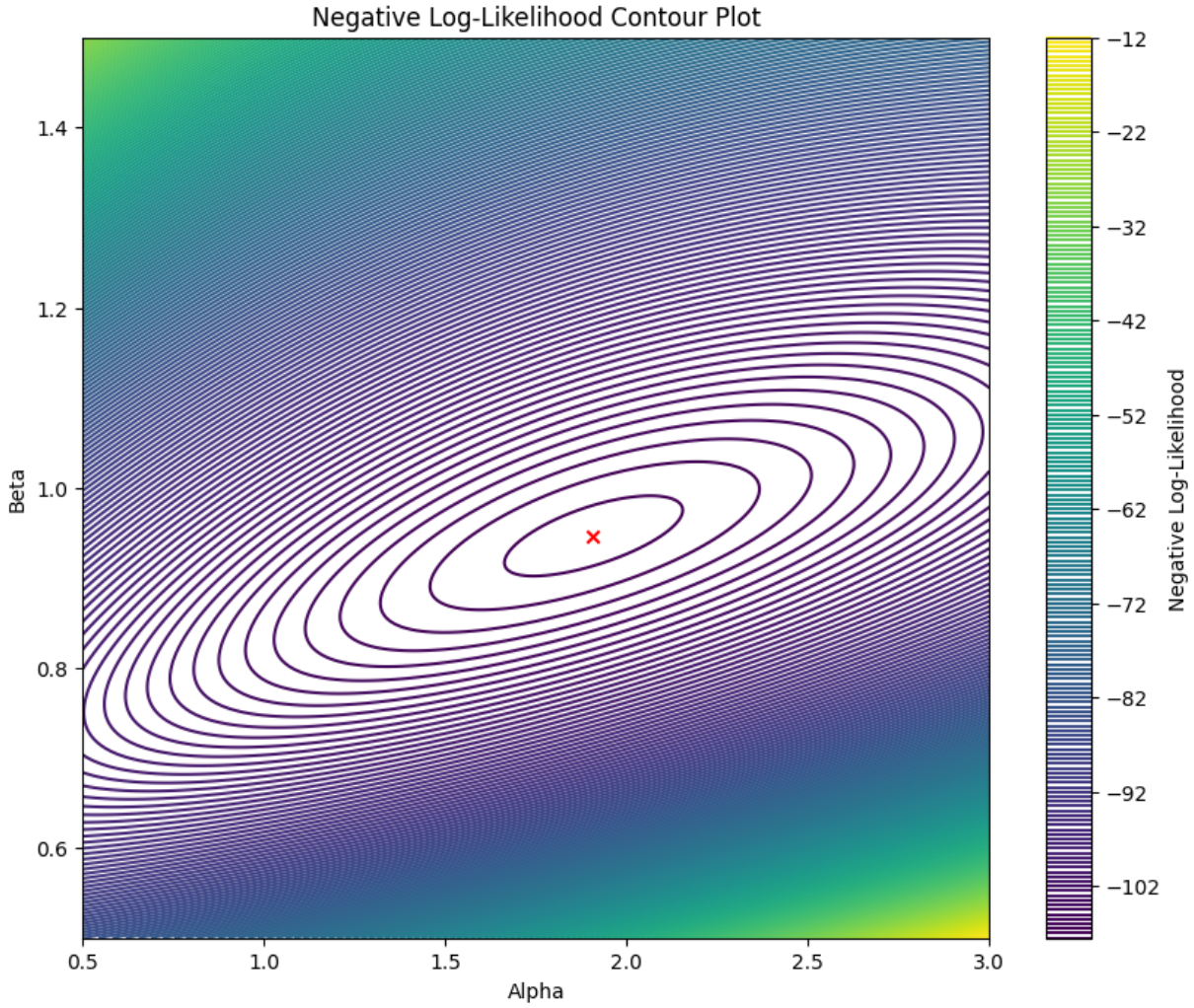
3. 繪製立體圖和等高線圖，以視覺化目標函數的形狀
4. 使用 Scipy 中的 `minimize` 函數來優化目標函數，找到最小化概似函數的參數值
5. 繪製等高線圖上的最小值點
6. 以立體圖的形式呈現目標函數在不同參數值下的形狀

- 計算： 最大概似估計 MLE α, β： $\max_{\alpha,\beta>0} \ln L(\alpha, \beta)$
- 聯合概似函數： $L(\alpha, \beta) = \prod_{i=1}^{n} \frac{f_t(v_i|\alpha,\beta)}{F_T(u_i|\alpha,\beta)^{-1}}$; $f_t(v|\alpha, \beta) = \alpha\beta v^{\beta-1} \exp(-\alpha v^{\beta})$; $F_T(u|\alpha, \beta) = 1 - \exp(-\alpha u^{\beta})$
- ft: 代表在給定 beta 的情況下，對 v 進行機率密度函數估計
- FT: 代表對 u 進行累積分佈函數估計
  - 這兩部分組合形成聯合概似函數 log_likelihood，其目標是通過最大化估計值，使模型與實際資料的概似性最大化函數的參數值
- 最後，輸出 MLE 結果，找到最佳參數值 alpha 和 beta
  - 可解釋為: 對於給定的資料，使用該模型的 alpha 和 beta 參數值 最有可能解釋觀測到的資料分佈
- `Negative Log-Likelihood`（負對數似然）
  - 在統計和機器學習中用於最大概似估計（Maximum Likelihood Estimation, MLE）的常見術語之一
  - 估計模型參數時最小化的目標函數之一
  - Log-Likelihood是似然的自然對數; 取負號即負對數似然，用於最小化目標函數，因為 最小化負對數似然等同於最大化似然

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.optimize import minimize

# 輸入檔案 --> 先獨立命名該檔案，在 load 它
data_dir = '../'  # 根據你的檔案路徑修改
D = np.loadtxt(data_dir + 'UV.txt', comments= '%', encoding='Latin-1')
u, v = D[:, 0], D[:, 1]  # 資料為兩行，兩行分別看成兩個不同向量 (u ,v)
n = D.shape[0]  # n*2 的資料大小 --> 一行有 n 個樣本數，共有 (u ,v) 兩行

# 步驟 2：推導目標函數 \ln L(\alpha, \beta)
def log_likelihood(params):
    alpha, beta = params
    ft = alpha * beta * v**(beta - 1) * np.exp(-alpha * v**beta)
    FT = 1 - np.exp(-alpha * u**beta)
    log_likelihood = np.sum(np.log(ft) - np.log(FT))
    return -log_likelihood

# 步驟 3：繪製立體圖和等高線圖
alpha_range = np.linspace(0.5, 3, 500)
beta_range = np.linspace(0.5, 1.5, 500)
alpha_grid, beta_grid = np.meshgrid(alpha_range, beta_range)
likelihood_values = np.zeros_like(alpha_grid)

for i in range(alpha_range.shape[0]):
    for j in range(beta_range.shape[0]):
        likelihood_values[i, j] = log_likelihood([alpha_grid[i, j], beta_grid

# 繪製等高線圖
plt.figure(figsize=(10, 8))
contour = plt.contour(alpha_grid, beta_grid, likelihood_values, levels=200, c
```

```
plt.colorbar(contour, label='Negative Log-Likelihood')

# 步驟 4：部署 minimize 函數
initial_guess = [1.8, 1]
result = minimize(log_likelihood, initial_guess, method='L-BFGS-B', bounds=[(

# 標註最小值位置
plt.scatter(result.x[0], result.x[1], marker='x', color='red', label='Minimum
plt.title('Negative Log-Likelihood Contour Plot')
plt.xlabel('Alpha')
plt.ylabel('Beta')

# 兩個不同角度的立體圖
fig, axes = plt.subplots(1, 2, figsize=(15, 10), subplot_kw={'projection': '3

titles = ['Elev=-174°, Azim=35°', 'Elev=-162°, Azim=-30°']
for i, ax in enumerate(axes.flat):
    ax.plot_surface(alpha_grid, beta_grid, likelihood_values, cmap='viridis',
    ax.set_xlabel('Alpha')
    ax.set_ylabel('Beta')
    ax.set_zlabel('Negative Log-Likelihood')
    ax.set_title(titles[i])

# 調整每個子圖的視角
axes[0].view_init(elev=-174, azim=35)
axes[1].view_init(elev=-162, azim=-30)

plt.tight_layout()
plt.show()

print("MLE 結果:")
print("alpha:", result.x[0])
print("beta:", result.x[1])
```




```
MLE 結果:
alpha: 1.9073437928672317
beta: 0.9464377980284867
```

- 無論從平面或立體圖，都能觀察出最大值的位置，不過平面圖較明顯，而立體圖更有益於觀察整個估計分布狀況，可以看到三維的座標軸中了解更多函數意義。
- 而圖片呈現的資訊也和實際輸出的計算結果大同小異
- 等高線圖和立體圖，可以視覺化Negative Log-Likelihood在不同參數值上的變化情況